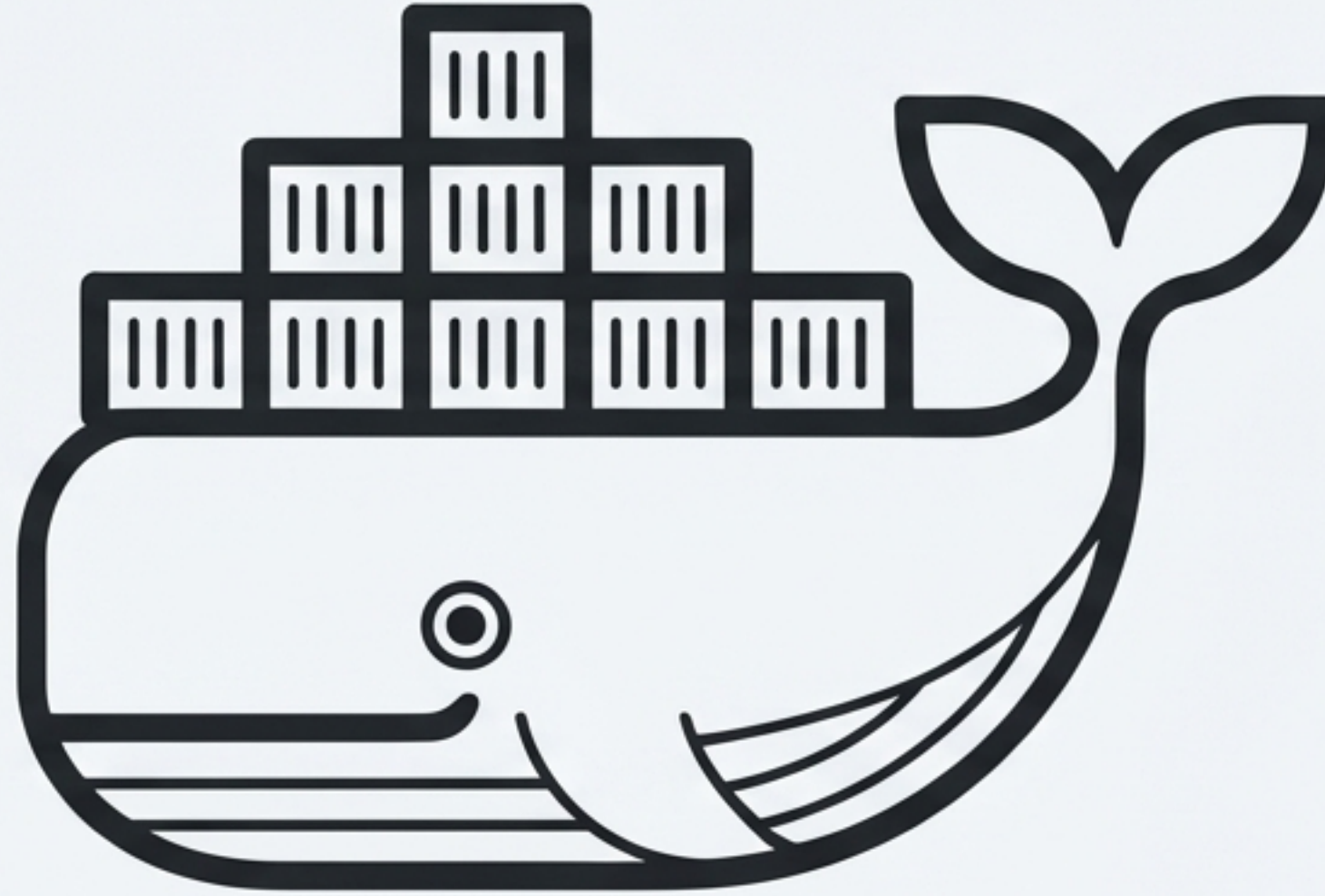




Fundamentos de Docker

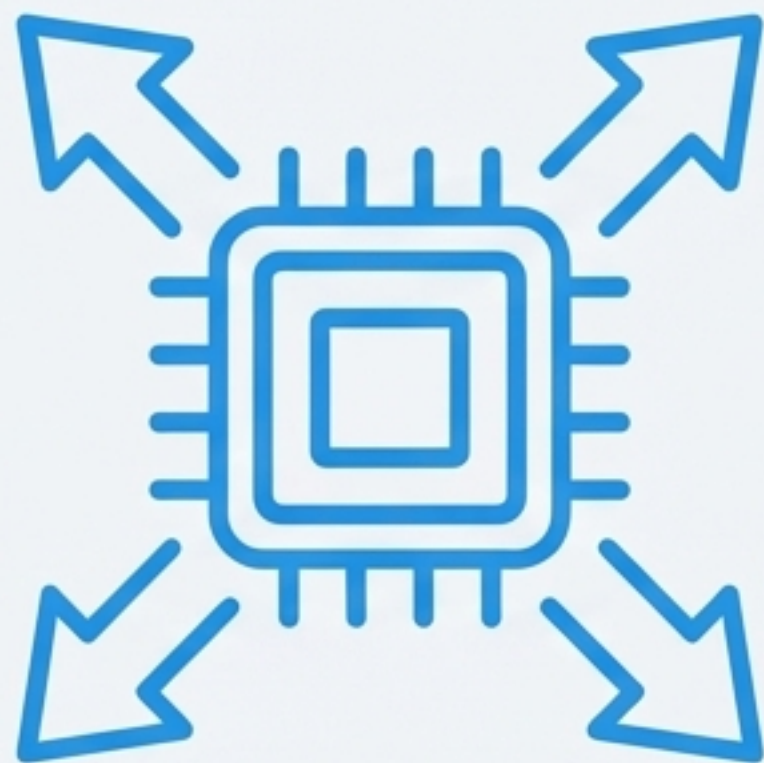
Da Virtualização Tradicional ao Setup do Ambiente Linux

O Dilema da Virtualização: Potência com Alto Custo

- **Sistema operacional completo** rodando dentro de um Host.
- **Hypervisor** gerencia recursos.
- **Guest OS independente**, garantindo o isolamento das aplicações.



As Desvantagens Críticas da Abordagem Tradicional



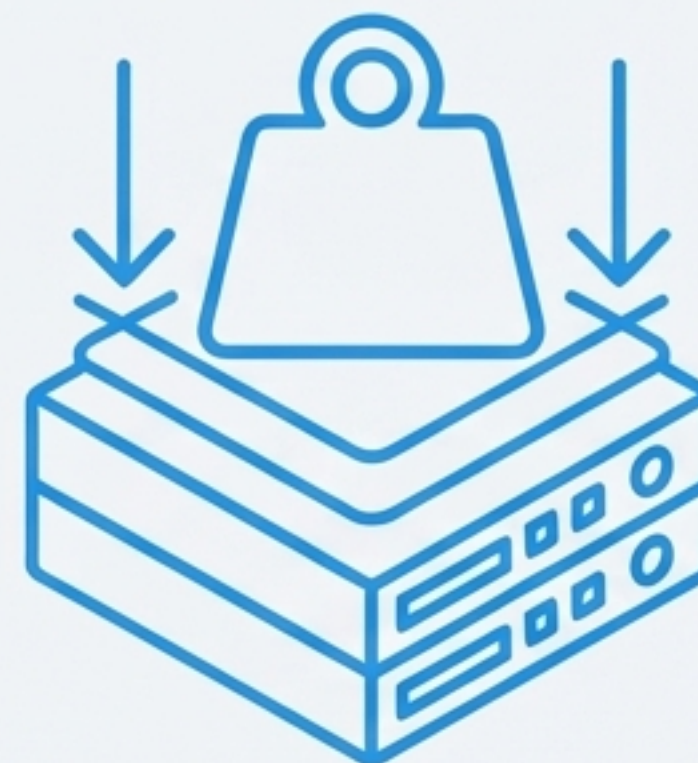
Alto Overhead

Múltiplos sistemas operacionais completos consomem recursos desnecessariamente.



Inicialização Lenta

Minutos para o boot de cada máquina virtual.



Consumo Pesado

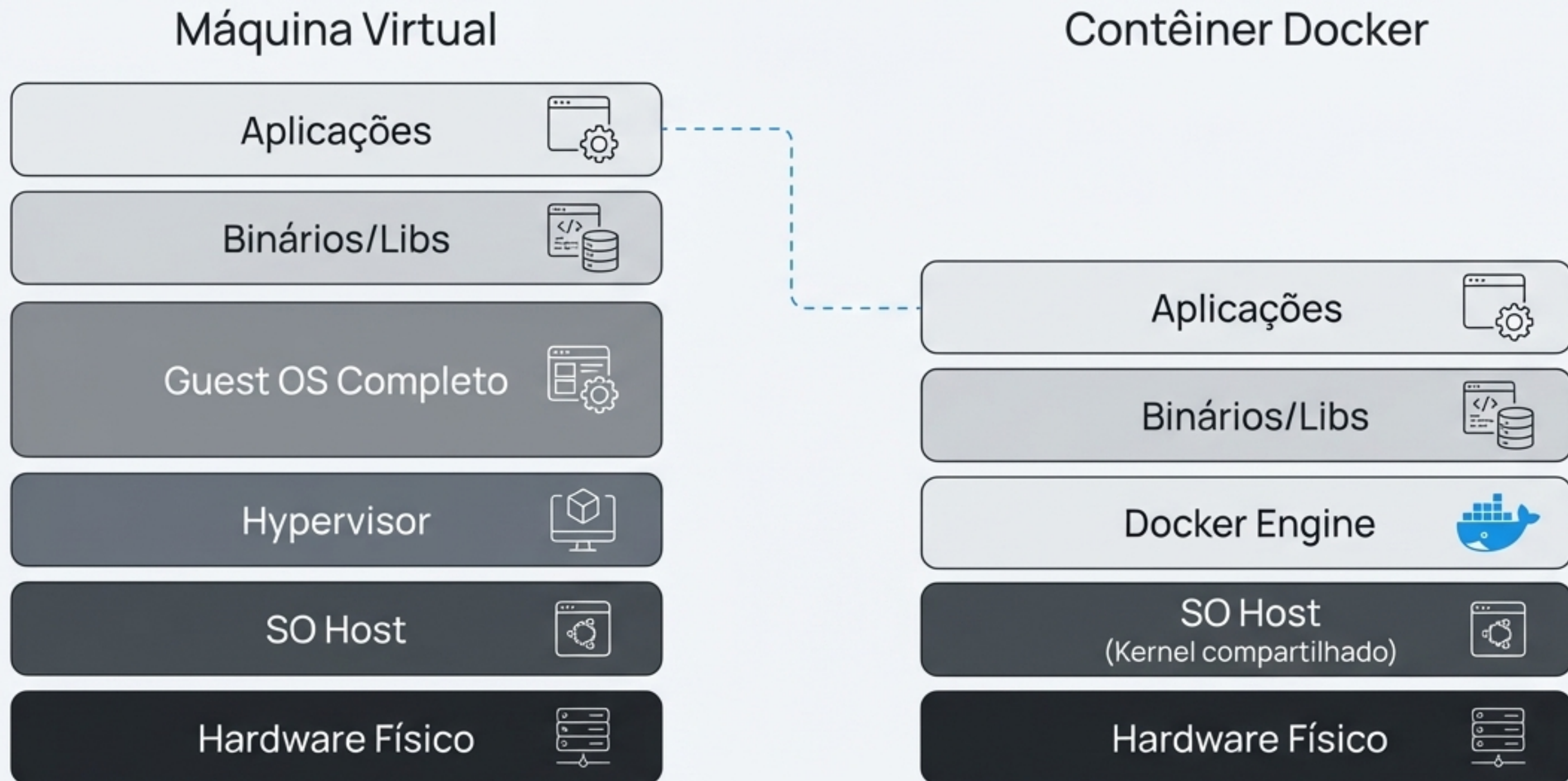
Alta demanda de CPU, RAM e espaço em disco para cada instância.

A Solução: A Revolução Leve e Rápida dos Contêineres

- **Processos Isolados:** Não é um SO completo; compartilha o **kernel** do Host.
- **Inicialização Rápida:** Sobe em **segundos**, não em minutos.
- **Extremamente Leves:** Inclui apenas os **binários** e **bibliotecas** estritamente necessários.
- **Alta Portabilidade:** Roda de forma **consistente** em qualquer ambiente com Docker.



VM vs. Contêiner: O Grande Contraste Arquitetural





Preparando o Terreno: O Ambiente Híbrido com WSL2

Por que WSL?

O Docker Engine é nativo do Linux. Para rodá-lo no Windows, precisamos de um kernel Linux.

WSL2 como Backend

Uma solução de virtualização leve que integra o Linux diretamente ao Windows, sem o overhead de uma VM tradicional.

Setup e Verificação do Ambiente WSL

Seção 1: Instalação

```
wsl --install
```

Este comando único baixa e instala a distribuição Linux padrão (Ubuntu) e o WSL2.



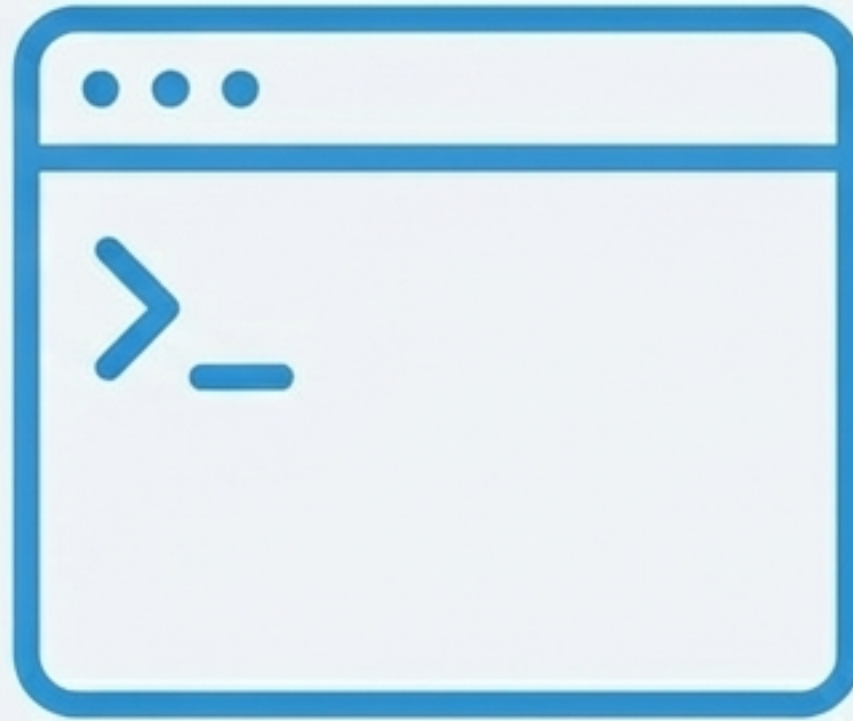
Seção 2: Verificação

```
wsl -l -v
```

Lista as distribuições Linux instaladas e verifica se estão rodando na versão 2 do WSL.

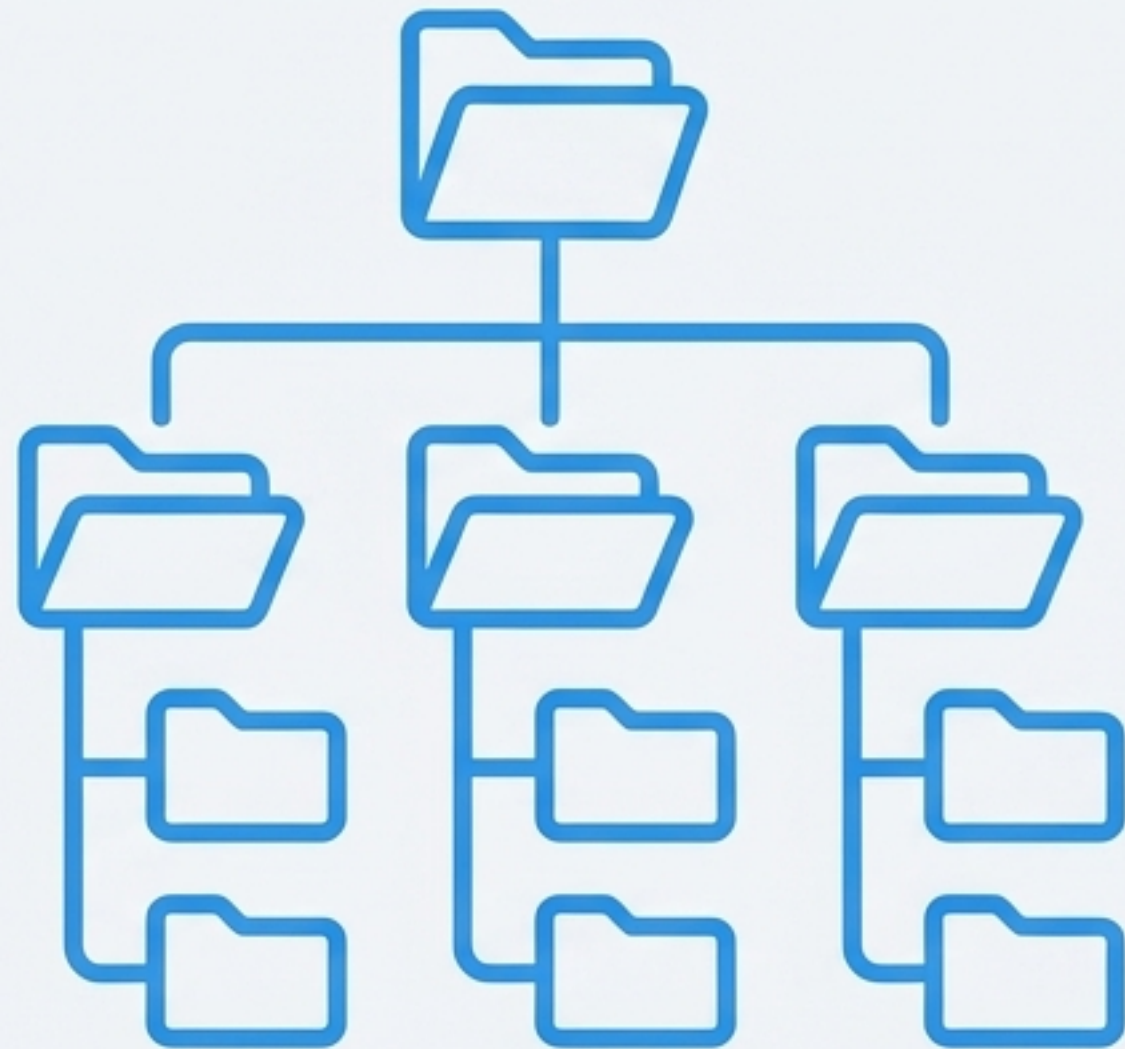


A Ferramenta Essencial: Domínio do Terminal Linux



O **domínio do terminal** é fundamental para trabalhar com Docker. Os comandos a seguir são a base para criar, gerenciar e depurar contêineres e seus ambientes.

Comandos Essenciais I: Navegação no Sistema



pwd (Print Working Directory)

Mostra o diretório (pasta) onde você está atualmente.

```
$ pwd
```

ls (List)

Lista os arquivos e diretórios no local atual.

```
$ ls -l
```

(a flag `-l` fornece mais detalhes)

cd (Change Directory)

Muda de um diretório para outro.

```
$ cd /home/usuario/documentos
```


Atalhos de Navegação: Movimente-se com Eficiência

Atalhos

~	Representa o diretório 'home' do seu usuário.
..	Refere-se ao diretório pai (sobe um nível).
.	Refere-se ao diretório atual.
/	Representa a raiz de todo o sistema de arquivos.

Exemplo Prático

```
# Navega para a pasta de logs
$ cd /var/log

# Confirma a localização atual
$ pwd
/var/log

# Retorna diretamente para a home
$ cd ~
```


Comandos Essenciais II: Criação e Manipulação



mkdir

Cria um novo diretório.

```
$ mkdir projeto
```



touch

Cria um arquivo vazio.

```
$ touch app.js
```



cat

Visualiza o conteúdo de um arquivo.

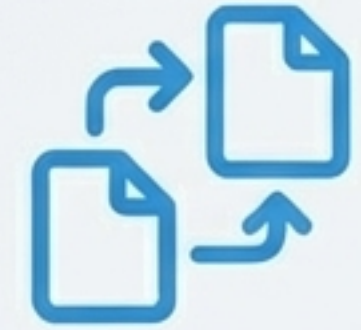
```
$ cat config.txt
```



echo

Escreve um texto. Usado com `>` ou `>>` para salvar em arquivos.

```
$ echo "OK" > log.txt
```



mv

Move ou renomeia um arquivo/diretório.

```
$ mv old.txt new.txt
```

Nota de Rodapé: Redirecionamento: `>` sobrescreve o conteúdo do arquivo, `>>` adiciona ao final.

Exemplo Prático: Um Fluxo de Trabalho Completo

```
# 1. Cria o diretório do nosso app
$ mkdir meu-app

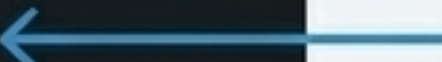
# 2. Entra no diretório
$ cd meu-app

# 3. Cria o arquivo principal
$ touch index.html

# 4. Adiciona conteúdo ao arquivo
$ echo "<h1>Hello Docker</h1>" > index.html

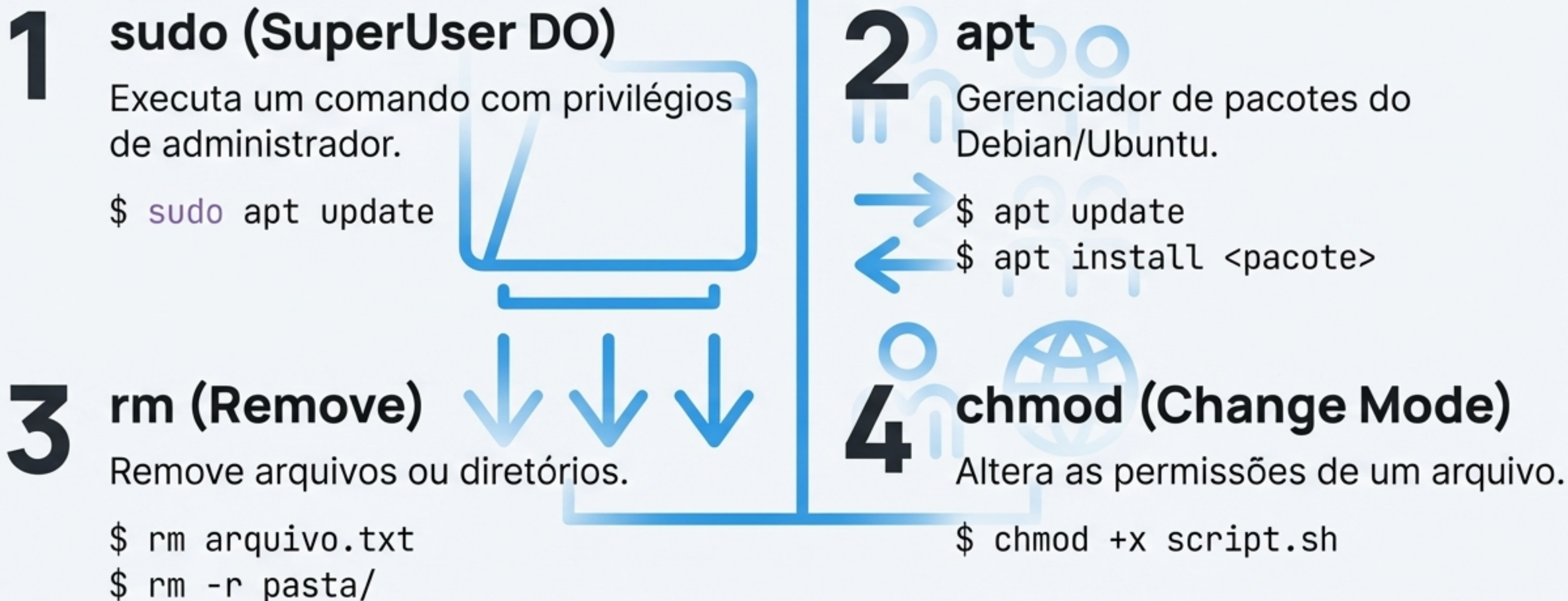
# 5. Verifica o conteúdo
$ cat index.html
<h1>Hello Docker</h1>

# 6. Renomeia o arquivo
$ mv index.html home.html
```



Este fluxo simples
simula a preparação
inicial de um projeto
web.

Comandos Essenciais III: Administração do Sistema



Decifrando Permissões no Linux: O Sistema `rwx`

Conceitos Fundamentais



`r` → **read** (leitura)



`w` → **write** (escrita)



`x` → **execute** (execução)

Níveis de Aplicação



As permissões são aplicadas para três níveis: o **dono** do arquivo, o **grupo** ao qual ele pertence, e **outros** usuários.

Exemplos Comuns

```
chmod 755 script.sh
```

(Permissão comum para scripts)

```
chmod +x deploy.sh
```

(Adiciona permissão de execução)

```
chmod -R 644 *.txt
```

(Aplica permissões de leitura/escrita para o dono e apenas leitura para outros, em todos os arquivos .txt)

Seu Kit de Ferramentas Linux: Resumo Essencial

Navegação

`pwd`

`ls`

`cd`

Manipulação

`mkdir, touch`

`cat, echo`

`mv`

Administração

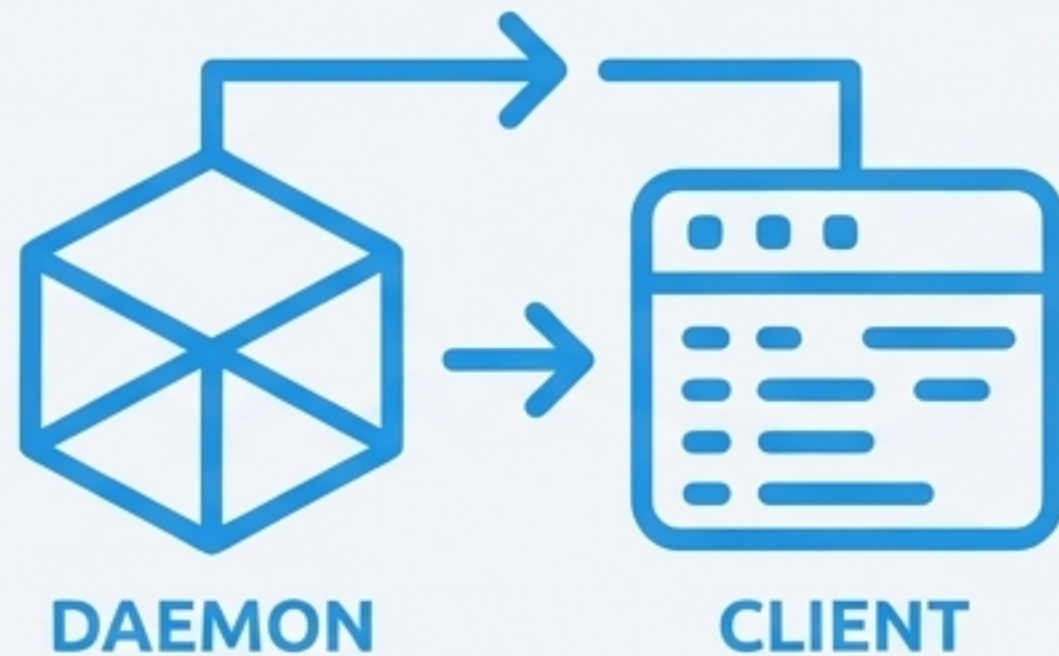
`sudo, apt`

`rm`

`chmod`

Pratique estes comandos. Eles serão a base do seu trabalho diário com Docker.

Próximos Passos: Mergulhando na Arquitetura Docker



O Que Vem a Seguir

- **Instalação do Docker:** Setup completo do Docker Engine no ambiente WSL.
- **Arquitetura Docker:** Entendendo os componentes: Docker Daemon, Client e Registry.
- **Seu Primeiro Contêiner:** Executando o ``docker run hello-world`` e vendo a mágica acontecer.

Prepare-se: a próxima aula é onde começamos a criar e gerenciar contêineres.